

DevPulse

PRD v3.0 — Agentic Orchestration Upgrade

DOCUMENT TYPE · Product Requirements Document

VERSION · v3.0 — Agentic Orchestration Upgrade

STATUS · Approved PRD Candidate

DATE · April 2026

SCOPE · Solo builder · Senior/Staff-leaning DS/AI · Agentic AI readiness

BASE · Extends DevPulse PRD v2.2 — all v2.2 content preserved

Production-simulated agentic developer change-intelligence system.

Extends v2.2 with: goal parsing — dependency delta detection —
deterministic task planning — multi-task execution over the existing
correctness-preserving pipeline — bounded recovery policies —
plan-level audit and aggregate verdict reporting.

Sidharth Kriplani

Solo-built · Non-production · Production-simulated · April 2026

Table of Contents

1. Why DevPulse v3 Exists
2. v3.0 Upgrade Rationale
3. What Is Preserved from v2.2
4. What Is Added in v3.0
5. Upgraded Positioning
6. Updated Architecture — Dual Entry Modes
7. New Agentic Orchestration Layer — 6 Components
8. New Schemas (v3.0 Additions)
9. New Evidence Artifacts (EA-25 to EA-32)
10. New Failure and Recovery Scenarios (F-11 to F-19)
11. Updated 30-Day Lifecycle — Extension Days 31–37
12. Updated Demo Narrative
13. Updated Must-Have / Should-Have / Optional / Never-Cut
14. Updated Evaluation Design — Agentic Layer
15. Updated Resume and Interview Positioning
16. Red-Team Review — v3.0
17. Recommendation Before Execution Planning

1. Why DevPulse v3 Exists

DevPulse v2.2 already proves that the builder can design retrieval-correct, version-aware, auditable AI systems. The 11-layer pipeline, 9 conflict types, SAFE/RISKY/BLOCKED verdict logic, LLM-last orchestration, 24 evidence artifacts, and 30-day simulated lifecycle all demonstrate the system discipline that traditional AI/ML hiring panels reward.

But AI-forward companies — pre-IPO, AI-native, and applied AI teams — increasingly expect builders who can go beyond answering questions. They want people who can build systems that pursue bounded goals through visible plan-act-verify-recover loops. DevPulse v3 adds this missing agentic layer without sacrificing the original correctness philosophy.

The upgrade is designed to show agentic readiness — not by letting an LLM act freely, but by orchestrating deterministic tools and evidence checks across multiple migration tasks in a bounded, auditable loop.

This upgrade improves the hiring signal for:

- AI-forward CEOs and CTO-level technical leaders
- AI-forward product and engineering managers at pre-IPO companies
- Applied AI and AI infrastructure teams
- Agentic AI / AI platform roles
- Senior IC roles where system design, traceability, and AI safety matter equally

Demonstrable new skills added:

- Agent architecture and goal decomposition
- Task planning and execution orchestration
- Deterministic recovery policies
- Multi-task audit traceability
- AI workflow productization
- AI-system ownership (not just AI usage)

What v3 does NOT do:

It does not let an LLM plan. It does not create an unconstrained autonomous agent. It does not write code, run tests, or open PRs. It orchestrates the existing DevPulse correctness infrastructure across multiple tasks in a bounded, policy-governed loop that any hiring reviewer can inspect and challenge.

2. v3.0 Upgrade Rationale

The v2.2 Hiring Signal Gap

DevPulse v2.2 is positioned for: Senior DS / Applied ML / Product DS / Platform / Senior IC hiring. What it does not currently demonstrate:

Missing Signal	Why It Matters
Goal-level task decomposition	AI-forward CEOs want builders who think in goal → plan → execute
Multi-step autonomous execution	Distinguishes decision-support from agentic systems
Recovery and retry logic	Production agentic systems fail and must recover gracefully
Visible agent loop	The loop must be inspectable and auditable, not opaque
Aggregate multi-dependency verdict	Real migrations span dozens of dependencies, not one query

The v3 Answer

v3 adds a bounded agentic orchestration layer on top of the existing v2.2 pipeline. The system now operates in two modes:

- **Query Mode (v2.2):** Single query → existing DevPulse pipeline → response / migration report.
- **Goal Mode (v3.0 NEW):** Goal + manifest → GoalParser → DependencyDeltaDetector → TaskPlanner → TaskExecutor loop → RecoveryDecider → PlanSummaryReporter.

The TaskExecutor in Goal Mode calls the existing v2.2 pipeline for every task. No new retrieval system. No new conflict detection. No new synthesis rules. The agentic layer orchestrates the correctness layer — it does not replace it.

Why This Is Not Fake Agent Branding

Fake Agent Pattern	DevPulse v3.0 Approach
LLM decides what to retrieve	Deterministic QueryParser + VersionExtractor decide
LLM decides recovery action	Rule-based RecoveryDecider with policy caps
LLM generates the plan	DependencyDeltaDetector + TaskPlanner use manifest + version delta logic
No visible loop	Full trace in agent_task_execution_trace.json
No retry caps	Explicit retry cap: max 2 retries per task; max 40% BLOCKED before goal-level halt
Claims autonomy, no evidence	Every step produces a verifiable JSON artifact

3. What Is Preserved from v2.2

All of the following are preserved exactly as specified in DevPulse PRD v2.2. This document does not modify them.

Preserved Architecture — All 11 Layers

- L1 Ingestion Pipeline · L2 Document & Chunk Store · L3 Query Parser

- L4 Version Extractor · L5 Complexity Router · L6 Retrieval Engine
- L7 Conflict Detector · L8 LLM Synthesizer · L9 Citation Assembler
- L10 Migration Decision Report · L11 Observability & Audit

Preserved Schemas

- documents, chunks (L2)
- query_audit_log, retrieval_traces, conflict_alerts, fallback_events (L11)

Preserved Evidence Artifacts

- EA-01 through EA-24 — all 24 artifacts, unchanged.

Preserved Failure Scenarios

- F-01 through F-10 — all 10 failure and recovery scenarios, unchanged.

Preserved 30-Day Lifecycle

- Days 1–30 — seed_devpulse.py lifecycle, unchanged.

Non-Negotiable Preservation Rules

1. **LLM-last rule** — The LLM must never own version filtering, conflict detection, citation assembly, source freshness checks, fallback decisions, or recovery policy. The LLM may only synthesise already-retrieved, already-validated evidence.
2. **Version correctness** — Wrong-version guidance remains the primary failure mode to prevent. Hard version pre-filtering must remain intact.
3. **Conflict detection** — All 9 conflict types remain deterministic and unchanged.
4. **SAFE / RISKY / BLOCKED verdict logic** — Migration verdicts remain evidence-driven and deterministic.
5. **Programmatic citations** — Citations are assembled from chunk metadata, not generated by the LLM.
6. **Auditability** — Every goal, task, retrieval, verdict, fallback, and recovery action must be logged.
7. **Production-simulated honesty** — No production user, traffic, or SLA claims. All claims are artifact-backed.
8. **Evidence-first design** — If a behavior cannot be demonstrated through a generated artifact, it is not claimed.

4. What Is Added in v3.0

New Components (6)

Component	Role
GoalParser	Parses a high-level migration goal + dependency manifest into a structured goal record
DependencyDeltaDetector	Identifies version deltas and classifies breaking change risk per dependency
TaskPlanner / TaskQueue	Converts parsed goal into an ordered, prioritised task queue

Component	Role
TaskExecutor	Runs each task through the existing v2.2 pipeline — no new retrieval or conflict logic
RecoveryDecider	Applies rule-based recovery policy for BLOCKED, ambiguous, or low-evidence outcomes
PlanSummaryReporter	Produces the final goal-level artifact: aggregate verdict, per-task trace, recommended migration order

New Schemas (5)

agent_goals · agent_tasks · agent_task_runs · recovery_actions · plan_summary_reports

New Evidence Artifacts (8)

EA-25 through EA-32

New Failure Scenarios (9)

F-11 through F-19 (includes F-19: Successful Recovery from Missing Evidence)

Lifecycle Extension

Days 31–37: Agentic Orchestration Extension Week

New Evaluation Metrics (10)

goal_parse_accuracy · dependency_extraction_accuracy · task_plan_completeness · task_order_correctness · recovery_policy_correctness · aggregate_verdict_correctness · trace_completeness_rate · unbounded_retry_rate · llm_planning_violation_rate · wrong_version_answer_rate (inherited)

5. Upgraded Positioning

DevPulse v3.0 is a production-simulated agentic developer change-intelligence system that preserves deterministic version correctness, conflict detection, grounded synthesis, auditability, and evidence artifacts while adding bounded multi-step goal execution over dependency migration tasks.

Hiring Signal Map (Updated)

Audience	Signal
Applied ML / DS hiring	Retrieval correctness, evaluation rigor, LLM bounding — unchanged
Platform / infra hiring	Operationally trustworthy, version-safe, fully auditable — unchanged
Product DS hiring	Scopes around real pain, defines success metrics — unchanged

Audience	Signal
Senior IC hiring	End-to-end: design, build, measure, operate, tradeoffs — unchanged
AI-forward CEO / CTO ★ NEW	Can build systems that pursue goals, not just answer queries
Agentic AI / AI platform ★ NEW	Goal decomposition, orchestration, recovery, trace visibility
Applied AI / pre-IPO ★ NEW	AI system ownership, not just AI usage

6. Updated Architecture — Dual Entry Modes

DevPulse v3.0 operates in two distinct entry modes. They share all underlying infrastructure and do not conflict.

QUERY MODE (v2.2)	GOAL MODE (v3.0 NEW)
Single migration query ↓ QueryParser VersionExtractor ComplexityRouter HybridRetrieval ConflictDetector MigrationReporter LLMSynthesizer* PostGenValidation CitationAssembler ResponseFormatter ↓ Migration Decision Report (per query)	Goal text + dependency manifest ↓ GoalParser DependencyDeltaDetector TaskPlanner / TaskQueue ↓ TaskExecutor (per task loop) ■ calls v2.2 pipeline for each task ↓ RecoveryDecider ↓ PlanSummaryReporter ↓ Plan Summary Report (goal-level)

Shared Infrastructure (v2.2 Unchanged)

L1–L11 Pipeline · BM25+pgvector+RRF · ConflictDetector · MigrationReporter · LLMSynthesizer* · CitationAssembler · query_audit_log · retrieval_traces · conflict_alerts · fallback_events · Langfuse · Sentry

The TaskExecutor does not bypass any v2.2 layer. For every task it executes: version filtering is enforced · conflict detection runs deterministically · citations are assembled programmatically · post-generation validation checks LLM output · every step is logged to the v2.2 audit schemas · task-level state is additionally logged to the v3.0 agentic schemas.

7. New Agentic Orchestration Layer — 6 Components

Component 1: GoalParser

Accepts a high-level migration goal and a dependency manifest, and produces a structured, deterministic goal record.

Inputs

- Goal text (e.g., "Assess migration safety for this repo from SDK v2 to SDK v3")
- **Must-have manifest formats (v3.0):** package.json, requirements.txt, or pasted manifest text
- **Optional manifest formats (v3.0 optional scope):** pyproject.toml, pom.xml
- Optional: explicit from_version, to_version, target dependency family, target ecosystem

Processing Rules

- Dependency names and versions are extracted deterministically from the manifest file.
- The LLM must not invent dependency versions. Unresolvable versions → version_status: unresolvable.
- Target versions come from: (a) explicit user input, (b) lock files, or (c) flagged as requires_evidence_lookup.
- Ambiguity flags are set when: target version is missing, manifest is partial, multiple version ranges exist.

Output Schema (agent_goals)

Field	Description
goal_id	UUID primary key
raw_goal_text	Original input text
manifest_type	package.json / requirements.txt / pyproject.toml / pom.xml / text
project_name	Extracted or inferred
ecosystem	npm / pip / maven / unknown
from_version_context / to_version_context	Explicit or inferred
parsed_status	complete / partial / ambiguous / failed
ambiguity_flag	Boolean + reason
dependency_list	Array of parsed dependency records
created_at	Timestamp

Component 2: DependencyDeltaDetector

Compares current dependency versions against target versions and classifies migration risk per dependency.

Controlled Demo Registry

In the v3.0 demo, target versions are supplied by a controlled config file: **dependency_target_registry.json**. This file stores known demo dependency versions deterministically, avoiding any dependency on live npm/PyPI registry calls and preventing hallucinated target versions.

Field	Description
dependency_name	Package name (e.g., auth-sdk)
ecosystem	npm / pip / maven
current_version	Version in demo manifest (e.g., 2.4.1)

Field	Description
target_version	Controlled demo target (e.g., 3.0.0)
criticality	high / medium / low
known_breaking_change	Boolean — true if major API surface changed
target_source	Always: controlled_demo_registry (not live registry)

dependency_target_registry.json is a controlled demo artifact, not a live package registry integration. It keeps the v3.0 demo deterministic and reproducible. DependencyDeltaDetector reads from this file when target_version is not supplied explicitly by the user.

Processing Rules

- Version delta type is determined deterministically from semver comparison.
- Breaking change risk is rule-based: major → high; minor with changelog breaking evidence → high; minor without → medium; patch → low.
- Target versions are resolved from: (a) explicit user input, (b) dependency_target_registry.json (controlled demo), or (c) flagged as requires_evidence_lookup.
- If target version is missing or ambiguous: breaking_change_risk: unknown, requires_migration_check: true.
- The LLM does not classify risk. Risk classification is deterministic.

Output per dependency

Field	Description
dependency_name	Package name
current_version	From manifest
target_version	Explicit / inferred / missing
version_delta_type	patch / minor / major / unknown
breaking_change_risk	low / medium / high / unknown
requires_migration_check	true / false
version_status	resolved / inferred / unresolvable

Component 3: TaskPlanner / TaskQueue

Converts the parsed goal into a deterministic, ordered task list.

Priority Ordering (deterministic)

Priority	Condition
1 (highest)	Major version jumps
2	Dependencies with breaking_change_risk = high
3	Dependencies flagged as critical runtime path (user-tagged)

Priority	Condition
4	Dependencies with missing/ambiguous target version
5	Minor version jumps
6 (lowest)	Patch version updates

Within the same priority tier, tasks are ordered alphabetically by dependency name for determinism.

Component 4: TaskExecutor

Runs each task through the existing DevPulse v2.2 pipeline in task-priority order. This is the central design constraint of v3.0: the agent does not reason freely. It orchestrates the existing correctness-preserving pipeline across multiple tasks.

Execution Steps per Task

1. Build migration query from task fields
2. Run QueryParser (v2.2 L3)
3. Run VersionExtractor (v2.2 L4)
4. Run ComplexityRouter → routes to MIGRATION path (v2.2 L5)
5. Run HybridRetrieval with version pre-filter (v2.2 L6)
6. Run ConflictDetector — 9 conflict types (v2.2 L7)
7. Run MigrationReporter → produces per-task SAFE/RISKY/BLOCKED verdict (v2.2 L10)
8. [Conditional] Run LLMSynthesizer only if: conflict_flag=False AND verdict!=BLOCKED AND LLM-last rule satisfied
9. Run PostGenValidation (v2.2 L8)
10. Run CitationAssembler (v2.2 L9)
11. Log task run to agent_task_runs · Update task_status in agent_tasks
12. If BLOCKED or ambiguous → hand off to RecoveryDecider

Layers that TaskExecutor never bypasses: version pre-filtering · conflict detection · post-generation validation · citation assembly · query_audit_log logging · retrieval_traces logging · conflict_alerts logging.

Component 5: RecoveryDecider

Handles task failure states deterministically. All recovery decisions are rule-based. The LLM does not make recovery decisions.

Recovery Policy Rules

Condition	Recovery Action
BLOCKED — missing to_version coverage	retry_with_cross_source_expansion (once)
BLOCKED — HIGH/CRITICAL conflict	skip_and_escalate — do not synthesise
BLOCKED — stale doc on critical path	retry_with_migration_guide_priority (once)

Condition	Recovery Action
BLOCKED — low evidence	retry_with_related_version_evidence (once)
Vector store unavailable	fallback_to_bm25_only — log degraded retrieval warning
Same task BLOCKED after retry cap	mark_escalated — stop retrying
LLM synthesis failed	Evidence-only verdict; overall agent run continues
LLM provider timeout	Provider fallback chain per v2.2 F-06

Retry Caps (non-negotiable)

- Maximum **2 retries per task** before marking escalated.
- If **≥ 40% of tasks are BLOCKED** → aggregate verdict becomes BLOCKED regardless of remaining tasks.
- If **any critical-priority (priority 1) task is BLOCKED** → aggregate verdict cannot be SAFE.

Recovery version-correctness rule: Recovery may expand source types, related version evidence, or migration-guide coverage — but it must never allow wrong-version evidence to produce a SAFE verdict. Exact requested-version coverage is still required for SAFE. `retry_with_related_version_evidence` expands evidence scope; it does not lower the version-correctness bar.

Component 6: PlanSummaryReporter

Creates the final goal-level artifact after all tasks have completed, been escalated, or skipped. This is the primary agentic evidence output of DevPulse v3.0.

The report must visibly show the full agent loop: Goal → Plan → Execute → Verify → Recover → Final Decision

Aggregate Verdict Logic

Condition	Aggregate Verdict
All tasks SAFE, no escalations	SAFE
Any task RISKY, none BLOCKED or escalated	RISKY
Any critical-priority task BLOCKED or escalated	BLOCKED
≥ 40% of tasks BLOCKED	BLOCKED
Any task escalated regardless of verdict	RISKY at minimum
Degraded retrieval warning present	RISKY at minimum

8. New Schemas (v3.0 Additions)

All five v3.0 schemas integrate with existing v2.2 schemas via foreign keys. `agent_task_runs.query_id` links directly to `query_audit_log.query_id`, preserving the full v2.2 audit trail per task.

Schema: agent_goals

Field	Type / Description
goal_id	UUID PRIMARY KEY
raw_goal_text	TEXT NOT NULL
manifest_type	VARCHAR(32) — package.json / requirements.txt / etc.
project_name	VARCHAR(128)
ecosystem	VARCHAR(32) — npm / pip / maven / unknown
from_version_context / to_version_context	VARCHAR(64)
parsed_status	VARCHAR(16) — complete / partial / ambiguous / failed
ambiguity_flag	BOOLEAN DEFAULT FALSE
ambiguity_reason	TEXT
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: agent_tasks

Field	Type / Description
task_id	UUID PRIMARY KEY
goal_id	UUID REFERENCES agent_goals(goal_id)
dependency_name	VARCHAR(128) NOT NULL
current_version / target_version	VARCHAR(64)
version_delta_type	VARCHAR(16) — patch / minor / major / unknown
breaking_change_risk	VARCHAR(16) — low / medium / high / unknown
priority	INTEGER NOT NULL
reason_for_priority	TEXT NOT NULL
task_status	VARCHAR(16) — pending / running / safe / risky / blocked / skipped / escalated
created_at / completed_at	TIMESTAMPTZ

Schema: agent_task_runs

Field	Type / Description
run_id	UUID PRIMARY KEY
task_id	UUID REFERENCES agent_tasks(task_id)
query_id	UUID REFERENCES query_audit_log(query_id) — links to v2.2 audit
route_taken	VARCHAR(16) — SIMPLE / HYBRID / MIGRATION

Field	Type / Description
retrieval_status	VARCHAR(16) — success / degraded / failed
conflict_flag	BOOLEAN DEFAULT FALSE
migration_verdict	VARCHAR(16) — SAFE / RISKY / BLOCKED
grounding_rate	NUMERIC(4,3)
retry_count	INTEGER DEFAULT 0
final_status	VARCHAR(16) — safe / risky / blocked / escalated / skipped
latency_ms	INTEGER
cost_usd	NUMERIC(8,6)
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: recovery_actions

Field	Type / Description
recovery_id	UUID PRIMARY KEY
task_id	UUID REFERENCES agent_tasks(task_id)
run_id	UUID REFERENCES agent_task_runs(run_id)
recovery_action	VARCHAR(64) — retry_with_cross_source_expansion / skip_and_escalate / etc.
reason	TEXT NOT NULL — human-readable trigger condition
retry_number	INTEGER NOT NULL — 1 or 2
outcome	VARCHAR(16) — success / still_blocked / escalated
capped_by_policy	BOOLEAN DEFAULT FALSE
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: plan_summary_reports

Field	Type / Description
report_id	UUID PRIMARY KEY
goal_id	UUID REFERENCES agent_goals(goal_id)
total_tasks / safe_tasks / risky_tasks / blocked_tasks	INTEGER
skipped_tasks / escalated_tasks	INTEGER DEFAULT 0
aggregate_verdict	VARCHAR(16) — SAFE / RISKY / BLOCKED
recommended_action	VARCHAR(64) — Proceed / Proceed with caution / Do not proceed
report_path	TEXT — path to plan_summary_report.json

Field	Type / Description
generated_at	TIMESTAMPTZ DEFAULT NOW()

9. New Evidence Artifacts (EA-25 to EA-32)

All 24 v2.2 evidence artifacts (EA-01 to EA-24) are preserved unchanged.

EA-25: agent_goal_parse_sample.json

Contents: Raw goal text, parsed manifest summary, full dependency list with current/target versions, version delta types, breaking_change_risk classifications, ambiguity flags, parsed_status.

How produced: GoalParser processes a sample manifest. seed_devpulse_v3.py on Day 31.

Interview signal: Shows deterministic manifest parsing. Proves target version inference is bounded and non-hallucinated. Ambiguity flags prove the system does not invent information.

EA-26: dependency_delta_report.json

Contents: Per-dependency table: dependency_name, current_version, target_version, version_delta_type, breaking_change_risk, requires_migration_check, version_status, delta_notes.

How produced: DependencyDeltaDetector runs over EA-25 output on Day 32.

Interview signal: Shows version delta classification without LLM. Proves system can triage dependency risk before retrieval.

EA-27: agent_task_plan.json

Contents: Ordered task queue with all agent_tasks records: task_id, dependency_name, versions, version_delta_type, priority, reason_for_priority, initial task_status (pending), created_at.

How produced: TaskPlanner runs over EA-26 output on Day 33.

Interview signal: Shows deterministic task ordering. Priority reasoning is explicit and inspectable. Proves task planning does not require an LLM.

EA-28: agent_task_execution_trace.json

Contents: For each completed task: task_id, dependency_name, migration query, query_id (v2.2 link), route_taken, retrieval_status, conflict_flag, conflict_types, migration_verdict, grounding_rate, retry_count, final_status, latency_ms, cost_usd.

How produced: TaskExecutor runs all tasks through the existing DevPulse pipeline on Day 34.

Interview signal: Most important v3.0 artifact. Proves agentic layer orchestrates the correctness pipeline. query_id links to v2.2 audit records. Demonstrates multi-task orchestration with per-task evidence.

EA-29: recovery_decision_log.json

Contents: All recovery_actions records: recovery_id, task_id, dependency_name, recovery_action, reason, retry_number, outcome, capped_by_policy, created_at. Summary counts per recovery action type.

How produced: RecoveryDecider generates during Day 35. Lifecycle injects BLOCKED-due-to-conflict and BLOCKED-due-to-missing-docs scenarios.

Interview signal: Proves recovery is deterministic and bounded. Demonstrates retry cap enforcement. Shows system does not loop indefinitely.

EA-30: plan_summary_report.json

Contents: Complete PlanSummaryReport: original_goal_text, manifest_summary, dependency_delta_table, ordered_task_plan, per_task_execution_trace, task counts, aggregate_verdict with reasoning, recommended_migration_order, do_not_proceed_blockers, recovery_summary, citations, generated_at.

How produced: PlanSummaryReporter aggregates all task run outputs on Day 36.

Interview signal: Primary agentic deliverable. Shows complete agent loop: Goal → Plan → Execute → Verify → Recover → Final Decision.

EA-31: agentic_eval_results.json

Contents: Evaluation results for the agentic layer. Per-metric: target acceptance gate, actual value (post-implementation), pass/fail. Includes sanity checks: unbounded_retry_rate = 0, llm_planning_violation_rate = 0.

How produced: Agentic eval harness runs on Day 37 over simulated goal runs from Days 31–36.

Interview signal: Proves agentic layer is evaluated, not just described. Target gates labeled as implementation acceptance thresholds — matches evidence-first philosophy.

EA-32: agentic_demo_report.txt

Contents: Terminal output of the full agentic loop end-to-end: goal → parsed manifest → dependency delta → ordered task plan → per-task execution → recovery decisions → final plan summary → aggregate verdict.

How produced: show_agentic_demo.py runs on Day 37 over deterministic seed data.

Interview signal: Walk-through demo artifact. A hiring reviewer can run from a fresh clone and see the entire v3.0 agentic loop in under 2 minutes.

Supporting Config Artifact

dependency_target_registry.json

Purpose: Stores controlled target versions for demo dependencies. Keeps v3 demo deterministic. Avoids dependency on live external package registries. Prevents the system from hallucinating target versions.

How produced: Manually authored as part of seed_devpulse_v3.py setup. Read by DependencyDeltaDetector on Day 32 when target_version is not supplied explicitly.

Interview signal: Demonstrates production-simulated discipline: the builder explicitly controls what 'target version' means in the demo rather than querying live registries. Proves version information is grounded, not hallucinated.

This is a controlled demo registry artifact, not a live package registry integration. target_source field is always 'controlled_demo_registry' — inspectable in EA-26.

10. New Failure and Recovery Scenarios (F-11 to F-19)

All 10 v2.2 failure scenarios (F-01 to F-10) are preserved unchanged.

F-11: Ambiguous Migration Goal

INJECT	Input goal text contains no target version ("Upgrade all SDKs to latest").
EXPECTED	GoalParser sets ambiguity_flag = true. Affected tasks created with version_status: unresolvable. No fake target version invented.
RECOVERY	DependencyDeltaDetector marks breaking_change_risk: unknown. TaskExecutor attempts cross-source expansion. If not found, task escalated.

F-12: Unsupported Manifest Format

INJECT	Input manifest is malformed JSON or unrecognised format.
EXPECTED	GoalParser sets parsed_status: failed. Agent goal marked BLOCKED immediately. No partial hallucinated plan.
RECOVERY	Error logged to agent_goals. User presented with failure reason and expected format.

F-13: Missing Target-Version Documentation

INJECT	Task has valid target version, but no corpus docs exist for that version.
EXPECTED	TaskExecutor returns BLOCKED (insufficient_version_coverage). RecoveryDecider applies retry_with_cross_source_expansion (once). Still missing → task escalated.
RECOVERY	recovery_actions log shows retry_number=1, action=retry_with_cross_source_expansion. If still_blocked, retry_number=2 → skip_and_escalate, capped_by_policy=true.

F-14: High-Severity Conflict During Task Execution

INJECT	ConflictDetector finds CRITICAL conflict (same_api_different_behavior) during task execution.
EXPECTED	Existing v2.2 conflict detection fires. conflict_alerts record created. Synthesis suppressed. Task verdict = BLOCKED. RecoveryDecider: skip_and_escalate (no retry).
RECOVERY	Task added to do_not_proceed_blockers. Aggregate verdict cannot be SAFE if task is critical-priority.

F-15: Recovery Loop Cap Reached

INJECT	Task is BLOCKED after retry 1 and BLOCKED again after retry 2.
EXPECTED	RecoveryDecider enforces retry cap. capped_by_policy = true. Task status = escalated. No third retry. retry_count = 2 in agent_task_runs.
RECOVERY	Task added to do_not_proceed_blockers. agentic_eval_results.json asserts unbounded_retry_rate = 0.

F-16: Mixed Aggregate Migration Verdict

INJECT	Goal run: 2 SAFE, 1 RISKY, 1 BLOCKED (low priority), 1 BLOCKED (critical priority — priority 1).
EXPECTED	Because one critical-priority task is BLOCKED, aggregate_verdict = BLOCKED. recommended_action = Do not proceed. Staged recommendation: SAFE tasks could proceed independently.
RECOVERY	This is correct aggregate verdict behaviour. EA-30 demonstrates this mixed-verdict scenario.

F-17: Vector Store Unavailable During Agent Execution

INJECT	pgvector killed mid-run after 2 tasks completed, during task 3.
EXPECTED	v2.2 F-05 fires at task level. Task 3 falls back to BM25-only. fallback_event logged. retrieval_status = degraded. Plan report includes degraded_retrieval_warning. Aggregate verdict = RISKY at minimum.
RECOVERY	Remaining tasks continue with BM25-only. Report documents which tasks ran under degraded retrieval.

F-18: LLM Synthesis Fails for One Task

INJECT	LLM provider times out on one task's synthesis call.
EXPECTED	v2.2 F-06 provider fallback fires. If all providers fail, synthesis suppressed. Task receives evidence-only deterministic verdict. grounding_rate = null. Agent run continues.
RECOVERY	Evidence-only verdict is valid. LLM failure does not block the goal run. synthesis_suppressed: true in agent_task_execution_trace.json.

F-19: Successful Recovery from Missing Evidence

INJECT	Initial retrieval for one task lacks target-version migration-guide coverage. First task run returns BLOCKED due to insufficient evidence (evidence_coverage_score below threshold).
EXPECTED	RecoveryDecider applies retry_with_related_version_evidence (retry 1). The retry finds a migration-guide chunk or related-version evidence sufficient for a caveated recommendation. Task verdict moves from BLOCKED to RISKY. Verdict cannot move to SAFE because exact full target-version coverage is still incomplete.
RECOVERY	recovery_decision_log.json shows retry action, trigger condition, and improved outcome. agent_task_execution_trace.json shows first verdict BLOCKED, second verdict RISKY, retry_count=1. plan_summary_report.json includes task as RISKY with caveats. This scenario proves RecoveryDecider is not merely an escalation logger — it can improve task state while respecting version-correctness boundaries.

11. Updated 30-Day Lifecycle — Extension Days 31–37

Days 1–30 from v2.2 are preserved unchanged. `seed_devpulse.py` generates the full v2.2 30-day lifecycle as specified.

A new script, `seed_devpulse_v3.py`, generates the agentic extension lifecycle. It depends on the v2.2 lifecycle having been run first.

Day 31: GoalParser + Manifest Parsing

- Ingest sample dependency manifest (5–7 deps: major, minor, patch)
- Run GoalParser over manifest + goal text
- Assert `parsed_status = complete` for well-formed input
- Assert `ambiguity_flag = true` when target version missing
- Assert no invented versions in `dependency_list`

Evidence artifact: EA-25 `agent_goal_parse_sample.json`

Exit gate: *`parsed_status = complete` AND no unresolvable versions marked as resolved*

Day 32: DependencyDeltaDetector + Controlled Registry

- Author `dependency_target_registry.json` with controlled demo target versions
- Run DependencyDeltaDetector over parsed goal (reads from controlled registry)
- Assert `version_delta_type` correctly classified for all deps
- Assert `breaking_change_risk = unknown` for missing target versions
- Assert `target_source = 'controlled_demo_registry'` in all resolved records
- Assert at least one dep classified as major / high risk

Evidence artifact: EA-26 `dependency_delta_report.json` · `dependency_target_registry.json` (supporting config)

Exit gate: *`version_delta_type` populated for all resolvable deps; risk classification matches semver logic; no live registry calls*

Day 33: TaskPlanner / TaskQueue

- Run TaskPlanner over EA-26 output
- Assert tasks ordered by priority (major version jumps first)
- Assert `reason_for_priority` populated for all tasks
- Assert at least one ambiguous task placed correctly in priority order

Evidence artifact: EA-27 `agent_task_plan.json`

Exit gate: *Task count matches dependency count; priority ordering is deterministic; all `task_status` = pending*

Day 34: TaskExecutor Loop

- Run TaskExecutor over the task queue from Day 33
- Assert query_id populated in agent_task_runs (links to query_audit_log)
- Assert at least one task: SAFE, one: RISKY, one: BLOCKED
- Assert retrieval_traces and conflict_alerts populated for tasks that triggered them
- Assert synthesis suppressed for BLOCKED tasks

Evidence artifact: EA-28 agent_task_execution_trace.json

Exit gate: All tasks have final_status; query_id populated per task; no task has retry_count > 0 at this stage

Day 35: RecoveryDecider — BLOCKED Scenarios

- Inject BLOCKED-due-to-missing-docs: assert retry_with_cross_source_expansion fires
- Inject BLOCKED-due-to-HIGH-conflict: assert skip_and_escalate (no retry)
- Inject retry-cap scenario: assert capped_by_policy = true after retry 2
- Assert unbounded_retry_rate = 0 (no task has retry_count > 2)

Evidence artifact: EA-29 recovery_decision_log.json

Exit gate: All recovery actions logged; capped_by_policy = true for tasks hitting cap; no retry_count > 2

Day 36: PlanSummaryReporter

- Run PlanSummaryReporter over all task outcomes from Days 34–35
- Assert plan_summary_report.json contains all required fields
- Assert aggregate_verdict logic correctly applied
- Assert do_not_proceed_blockers lists all escalated/BLOCKED critical-priority tasks
- Assert citations aggregated from all task runs

Evidence artifact: EA-30 plan_summary_report.json

Exit gate: aggregate_verdict non-null; do_not_proceed_blockers populated when critical task BLOCKED

Day 37: Agentic Demo + Evaluation

- Run full agentic eval harness over Days 31–36 outputs
- Assert all 10 agentic eval metrics meet target gates
- Assert unbounded_retry_rate = 0; llm_planning_violation_rate = 0
- Run show_agentic_demo.py end-to-end; capture terminal output

Evidence artifact: EA-31 agentic_eval_results.json · EA-32 agentic_demo_report.txt

Exit gate: show_agentic_demo.py runs in under 3 minutes; all agentic eval thresholds met; all 8 v3.0 artifacts present

12. Updated Demo Narrative

30-Second Pitch

*DevPulse is an agentic developer change-intelligence system. You give it a dependency manifest and a migration goal — and it decomposes the goal into an ordered migration task queue, runs each dependency through a version-aware retrieval and conflict-detection pipeline, applies deterministic recovery policies when evidence is missing or conflicts are found, and produces an auditable migration plan with a **SAFE**, **RISKY**, or **BLOCKED** aggregate verdict. Every step is logged. The agent does not reason freely — it orchestrates correctness-preserving tools across multiple tasks.*

2-Minute Architecture Walkthrough

DevPulse has two entry modes. In Query Mode — the v2.2 core — a single migration query flows through an 11-layer pipeline: parsing, version extraction, hybrid BM25+pgvector+RRF retrieval with hard version pre-filtering, deterministic conflict detection across 9 conflict types, **SAFE/RISKY/BLOCKED** migration verdict, conditional LLM synthesis with post-generation grounding validation, and programmatic citation assembly.

In Goal Mode — the v3.0 agentic layer — a dependency manifest and goal text enter a GoalParser, which extracts structured dependency records without inventing version information. A DependencyDeltaDetector classifies version deltas and breaking-change risk. A TaskPlanner creates a priority-ordered task queue — major version jumps first. A TaskExecutor loops over the queue, calling the existing v2.2 pipeline for every task. When a task returns **BLOCKED**, a rule-based RecoveryDecider applies a bounded retry policy. Finally, a PlanSummaryReporter aggregates all outcomes into a goal-level **SAFE/RISKY/BLOCKED** verdict with a recommended migration order and a list of blockers that must be resolved before proceeding.

The agentic layer does not replace the correctness layer. It orchestrates it.

5-Minute Demo Walkthrough

- [1] Show EA-25 (agent_goal_parse_sample.json): 5-dep manifest, parsed goal, ambiguity flag on dep with missing target.
- [2] Show EA-26 (dependency_delta_report.json): two major/HIGH, one minor/MEDIUM, one patch/LOW, one unknown.
- [3] Show EA-27 (agent_task_plan.json): priority 1 tasks first, reason_for_priority populated for each.
- [4] Show EA-28 (agent_task_execution_trace.json): Task 1 = **RISKY**; Task 2 = **BLOCKED** (CRITICAL conflict, synthesis suppressed); Task 3+4 = **SAFE**; Task 5 = **BLOCKED** (missing evidence).
- [5] Show EA-29 (recovery_decision_log.json): Task 2 → skip_and_escalate (CRITICAL conflict, no retry). Task 5 → initially **BLOCKED** (missing target-version evidence) → RecoveryDecider applies retry_with_related_version_evidence → second run finds related migration-guide evidence → task improves from **BLOCKED** to **RISKY** with caveats. retry_count=1. No **SAFE** verdict issued because exact full target-version coverage is still incomplete. This proves RecoveryDecider can improve task state, not only log escalations.
- [6] Show EA-30 (plan_summary_report.json): aggregate_verdict = **BLOCKED** because critical-priority dependency (payment-sdk) remains **BLOCKED** and escalated. Staged recommendation: **SAFE** tasks (analytics-sdk, logging-lib) can proceed independently; Task 5 (unknown-dep) can proceed only as **RISKY** with caveats and reviewer approval; payment-sdk must be resolved before full migration proceeds.

- [7] Show EA-31 (agentic_eval_results.json): all metrics at gates, unbounded_retry_rate = 0, llm_planning_violation_rate = 0.
- [8] Run show_agentic_demo.py live: full terminal output in under 3 minutes.
- [9] Explain v4 scope: repo inspection, code patches, test execution, PR generation — out of scope for v3 intentionally.

13. Updated Must-Have / Should-Have / Optional / Never-Cut

MUST-HAVE (v3.0 Additions)

- GoalParser with deterministic manifest parsing and ambiguity flagging
- Manifest support: package.json, requirements.txt, pasted manifest text
- dependency_target_registry.json (controlled demo registry — prevents hallucinated target versions)
- DependencyDeltaDetector with semver-based delta classification
- TaskPlanner / TaskQueue with deterministic priority ordering
- TaskExecutor loop over existing DevPulse v2.2 pipeline (no new retrieval or conflict logic)
- RecoveryDecider with rule-based recovery policy and retry caps
- PlanSummaryReporter with aggregate verdict logic, staged migration recommendation, and do_not_proceed_blockers
- All five v3.0 agentic audit schemas
- EA-25 through EA-32 (all 8 v3.0 evidence artifacts)
- Agentic eval harness with 10 target acceptance gates
- seed_devpulse_v3.py generating Days 31–37 lifecycle
- show_agentic_demo.py producing EA-32

SHOULD-HAVE (v3.0)

- Dependency criticality tagging (user-supplied critical runtime path flag)
- Richer task ordering policy incorporating user-supplied criticality tags
- Cost and latency reporting at goal level (sum of per-task costs)

OPTIONAL (v3.0)

- pyproject.toml and pom.xml manifest support
- Repo inspection (scan actual repo files for dependency usage)
- Code patch proposal generation
- Test execution integration
- PR generation / GitHub / GitLab integration
- Slack / Linear integration for migration plan notifications
- Web-based plan summary report UI

NEVER-CUT (v3.0 Additions)

- LLM-last rule (LLM never plans, routes, recovers, or classifies risk)
- Version correctness (wrong-version answer rate = 0 in all task runs)
- Deterministic recovery (RecoveryDecider is rule-based, never LLM-driven)
- No unbounded loops (retry_count capped at 2 per task)
- Trace visibility (EA-28 and EA-32 must show the full agent loop)
- Audit artifacts (agent_task_runs links every task to v2.2 query_audit_log)
- Aggregate verdict correctness (F-16 must produce BLOCKED correctly)

Explicit Out of Scope for v3.0

- Agent writes code patches or modifies repo files
- Agent runs tests or opens PRs
- Autonomous deployment or live GitHub integration
- LLM-driven planning or recovery decisions
- Unbounded autonomous loops
- Production traffic or real user claims

The above may be mentioned as v4/future extensions, not v3 scope.

14. Updated Evaluation Design — Agentic Layer

All v2.2 evaluation criteria (10 categories, 50+ queries, 8 operational metrics) are preserved unchanged. The following agentic metrics are added. All values are **target acceptance gates** — not claimed until EA-31 exists.

Metric	Definition	Target Gate	Priority
goal_parse_accuracy	Goals correctly parsed (all deps extracted, no invented versions) / total	≥ 0.95	CRITICAL
dependency_extraction_accuracy	Deps correctly extracted / total deps in manifest	≥ 0.98	CRITICAL
task_plan_completeness	Tasks created / deps requiring migration check	$= 1.0$	HIGH
task_order_correctness	Task plans with correct priority ordering / total plans	≥ 0.95	HIGH
recovery_policy_correctness	Recovery actions matching policy rules / total recovery events	$= 1.0$	CRITICAL
aggregate_verdict_correctness	Aggregate verdicts matching expected / total goal runs	$= 1.0$	CRITICAL
trace_completeness_rate	Task runs with all required fields populated / total runs	≥ 0.98	HIGH
unbounded_retry_rate	Tasks with retry_count > 2 / total tasks	$= 0.0$	CRITICAL
llm_planning_violation_rate	Tasks where LLM was used for planning, routing, or recovery / total	$= 0.0$	CRITICAL
wrong_version_answer_rate	Inherited from v2.2 CAT-2; applies to all task-level retrieval	$= 0.0$	CRITICAL

15. Updated Resume and Interview Positioning

Resume Bullets (use 2 of 3)

- 1 Built DevPulse v3, a production-simulated agentic developer change-intelligence system that accepts dependency migration goals, decomposes them into ordered migration tasks, runs each task through a version-aware hybrid BM25+pgvector+RRF retrieval and conflict-detection pipeline, applies deterministic recovery policies for blocked cases, and produces auditable SAFE/RISKY/BLOCKED aggregate migration plans — with goal parse accuracy, aggregate verdict correctness, and trace completeness captured in `agentic_eval_results.json` and `plan_summary_report.json` after implementation.
- 2 Designed a 6-component agentic orchestration layer (GoalParser → DependencyDeltaDetector → TaskPlanner → TaskExecutor → RecoveryDecider → PlanSummaryReporter) over a production-simulated retrieval pipeline, with 5 agentic audit schemas, 8 evidence artifacts, and a 37-day simulated operating lifecycle — preserving LLM-last orchestration, hard version filtering, and deterministic conflict detection from the v2.2 core system.

- 3** Extended DevPulse with bounded agentic recovery policies (retry cap = 2 per task, 40% blocked = goal-level halt) and mixed-verdict aggregate logic; agentic eval harness asserts `unbounded_retry_rate = 0` and `llm_planning_violation_rate = 0` — with results captured in `agentic_demo_report.txt` after implementation.

Interview Q&A;

Q: Why is this agentic and not just batch processing?

A: Batch processing applies the same fixed operation to all inputs. DevPulse v3 is agentic because the system makes decisions during execution that affect subsequent steps. The RecoveryDecider changes the retrieval strategy based on what the ConflictDetector found. The TaskPlanner orders tasks based on risk signals from the DependencyDeltaDetector. The aggregate verdict is computed dynamically from task outcomes. The loop responds to its own outputs — that is the definition of agentic behaviour.

Q: Why not let the LLM plan?

A: The LLM-last rule exists because version correctness cannot be probabilistic. If an LLM plans the task sequence, it may create tasks for the wrong version, infer non-existent dependencies, or prioritise based on hallucinated risk signals. Every planning decision in DevPulse v3 is deterministic: manifest parsing extracts only what is in the file; version deltas are computed from semver comparison; task priority follows explicit rules. The LLM only synthesises already-validated evidence. LLM planning would violate the core correctness guarantee.

Q: How do you prevent wrong-version migration advice?

A: The same way v2.2 does, applied at every task. The version pre-filter in L4/L6 ensures task-level retrieval returns only chunks matching the task's `from_version` or `to_version`. The ConflictDetector checks for `version_deprecation_conflict` and `same_api_different_behavior`. The MigrationReporter requires complete version coverage before issuing a SAFE verdict. `wrong_version_answer_rate = 0.0` is a non-negotiable gate inherited from v2.2.

Q: What happens when one dependency is BLOCKED?

A: It depends on that dependency's priority. If it is priority 1, the aggregate verdict cannot be SAFE regardless of other task outcomes. PlanSummaryReporter adds it to `do_not_proceed_blockers`. The `recommended_migration_order` lists SAFE and RISKY dependencies as potentially proceeding independently, with a staged migration note.

Q: How does recovery work?

A: RecoveryDecider applies a rule table deterministically. The trigger condition determines the recovery action: missing docs → retry with cross-source expansion; HIGH/CRITICAL conflict → skip and escalate immediately (no retry); vector store down → BM25-only fallback. No decision is made by the LLM.

Q: How do you prevent infinite loops?

A: Explicitly, with policy caps. Each task has a maximum `retry_count` of 2. If still `BLOCKED` after two retries, `capped_by_policy` = true and the task is escalated. At the goal level, if $\geq 40\%$ of tasks are `BLOCKED`, the entire goal run is halted. The agentic eval harness asserts `unbounded_retry_rate` = 0 — verifiable from EA-31.

Q: Why did you add this v3 upgrade?

A: DevPulse v2.2 already demonstrates retrieval correctness, evaluation rigor, and production-simulated operational thinking. What it does not demonstrate is the ability to build a system that pursues goals through a visible plan-act-verify-recover loop. AI-forward companies need builders who understand how to make AI systems autonomous in a bounded and trustworthy way. The v3 upgrade adds that signal without compromising the correctness philosophy.

Q: How does this appeal to AI-forward CEOs and applied AI teams?

A: AI-forward CEOs are not looking for people who can use AI tools. They are looking for people who can build AI systems that reduce human effort at scale in a way that is inspectable and trustworthy. DevPulse v3 demonstrates exactly that: a system that takes a messy input, structures it, executes a multi-step correctness-preserving plan, handles failures without human intervention where possible, and escalates when it cannot — producing an auditable output that an engineering team can act on.

16. Red-Team Review — v3.0

Risk: The agentic layer is cosmetic — just a loop over v2.2 queries

The differentiation comes from: (a) goal decomposition before retrieval, (b) deterministic task ordering based on risk, (c) rule-based recovery that changes retrieval strategy mid-run, (d) aggregate verdict logic synthesising outcomes across tasks, (e) the visible trace artifact. Without all five, it is batch processing.

Mitigation: EA-30 must show at least one recovery action that changed the retrieval strategy AND produced a different outcome than the first attempt. Without a real recovery success story, the recovery layer is untestable.

Risk: GoalParser is just JSON parsing — not meaningfully agentic

The differentiation is the ambiguity detection: flagging missing target versions, partial manifests, unrecognised ecosystems, and unresolvable version strings — without inventing anything.

Mitigation: EA-25 must include at least one dep with missing target version flagged correctly AND one well-formed dep extracted correctly. The contrast is the signal.

Risk: Aggregate verdict logic is trivial

If aggregate verdict is just 'any `BLOCKED` = `BLOCKED`,' it does not demonstrate nuanced judgment. The staged migration recommendation is what elevates this.

Mitigation: EA-30 must show a mixed-verdict run with a staged recommendation. F-16 is the failure scenario that proves this works correctly.

Risk: Rule-based recovery is not impressive

LLM-driven recovery is actually worse for version-sensitive systems because it may bypass version constraints. Deterministic recovery is a feature: bounded, inspectable autonomy over free-form LLM judgment.

Mitigation: The interview Q&A; (Section 15) prepares this argument. recovery_policy_correctness = 1.0 in EA-31 proves the policy was applied correctly every time.

Risk: The v3 upgrade makes the portfolio feel bloated

v3 is an extension of an existing project, not a new project. The portfolio remains 4 pillars. DevPulse becomes deeper, not a 5th project.

Mitigation: The portfolio cover document should describe DevPulse as having two modes: decision-support (v2.2) and agentic orchestration (v3.0). One project, two signals.

What to Cut If Time Gets Tight (v3.0)

Cut in this order: (1) pyproject.toml / pom.xml manifest support, (2) dependency criticality tagging, (3) goal-level cost/latency reporting, (4) agentic eval harness (manual verification instead), (5) show_agentic_demo.py (present EA-28 and EA-30 directly).

Do NOT cut staged migration recommendation. The PlanSummaryReporter must include at minimum: (a) SAFE tasks can proceed independently; (b) RISKY tasks can proceed only with caveats/reviewer approval; (c) BLOCKED critical-priority tasks prevent full migration; (d) aggregate verdict cannot be SAFE if any critical task is BLOCKED or escalated. This is central to PlanSummaryReporter's interview value and the F-16 proof scenario.

The irreducible v3.0 core: GoalParser → DependencyDeltaDetector → TaskPlanner → TaskExecutor (v2.2 pipeline) → RecoveryDecider (capped retries) → PlanSummaryReporter → EA-28 + EA-30. Without this core producing verifiable artifacts, the v3 upgrade is not credible.

17. Recommendation Before Execution Planning

A separate execution plan will detail day-by-day tasks and milestones. The following covers high-level build sequencing and rationale.

Pre-condition: DevPulse v2.2 must be fully implemented and all 24 evidence artifacts (EA-01 to EA-24) must be present before beginning v3.0 development. seed_devpulse.py must run end-to-end before seed_devpulse_v3.py is built.

Week 5 (Days 31–33): Goal Parsing + Task Planning

Focus: GoalParser, DependencyDeltaDetector, TaskPlanner. Schemas: agent_goals, agent_tasks.

Exit gate: EA-27 exists and task priority ordering is correct. No invented versions in EA-25.

Week 6 (Days 34–35): Task Execution + Recovery

Focus: TaskExecutor loop + RecoveryDecider. Schemas: agent_task_runs, recovery_actions.

***Exit gate:** EA-28 exists with query_id populated per task (linking to v2.2 audit log). EA-29 shows at least one recovery action with correct policy applied. unbounded_retry_rate = 0 asserted.*

Week 7 (Days 36–37): Aggregation + Demo + Eval

Focus: PlanSummaryReporter, agentic eval harness, show_agentic_demo.py. Schema: plan_summary_reports.

***Exit gate:** EA-30 shows aggregate_verdict logic applied correctly over a mixed-outcome goal run. EA-31 shows all agentic eval gates met. EA-32 runs end-to-end in under 3 minutes from fresh seed.*

Final Positioning Statement

DevPulse v3.0 is not a generic agentic AI demo. It is a production-simulated agentic developer change-intelligence system that extends a correctness-preserving retrieval pipeline with bounded multi-step goal execution. Its agentic value comes from decomposing migration goals into ordered tasks, orchestrating version-correct evidence retrieval across all tasks, applying deterministic recovery policies when evidence is insufficient or conflicts are found, and producing aggregate SAFE/RISKY/BLOCKED migration plans with full execution traces. The 37-day simulated lifecycle, 32 evidence artifacts, and agentic evaluation suite distinguish it from both a portfolio prototype and a generic agent demo.

Honest Boundary Statement (Updated)

Does NOT Claim	Does Claim
Real enterprise deployment	Serious system design with justified choices
Private production data	Controlled simulation with realistic operating conditions
Live users or real traffic	Reproducible artifacts from a fresh environment
Autonomous production decisions	Bounded agentic execution with deterministic recovery
LLM-driven planning or recovery	Deterministic goal decomposition and rule-based recovery
Unbounded autonomous loops	Retry-capped, policy-governed task execution
Regulatory approval or MRM sign-off	Production-inspired operating discipline with auditability